Read expressions before output.

OOP LAB / THE REVISION ATLAS

Check validity first

Types & access → lifetime & bounds → selected function → output

Read after the notes

C++17 examples. Use this atlas to recall distinctions and short traces.

Instructor exam focus: "array is a constant pointer". The intended cue is a fixed array name with mutable elements. Precisely, an array has its own type and often converts to a pointer to its first element. Know both the cue and the exact C++ rule.

01 Expressions, types and conversions

Trace the state

STATEMENT	EXPRESSION RESULT	STORED x
int x = 2;	initial value	2
first = x++;	first gets 2	3
next = ++x;	next gets 4	4

Keep the expression result separate from the value stored after the statement.

- Zero is false; any nonzero numeric value is true, including a negative value.
- Postfix supplies the old value; prefix supplies the updated value. Track the expression result and final variable separately.
- Integer division happens before assignment: `double r=5/2;` stores 2. Convert an operand first to obtain 2.5: `static_cast<double>(5)/2`.
- In-range floating-to-int conversion truncates toward zero. `int(-4.8)` is -4.
- `sizeof(char)==1` is guaranteed. Other example sizes need an explicit platform assumption.

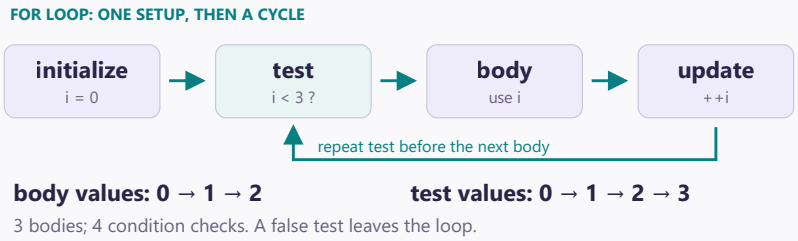
C++ / WORKED FRAGMENT

```
1 int x=2;
2 int a=x++ + 3; // a=5, x=3
3 int b=++x + 3; // b=7, x=4
```

Choose the path and the element.

02 Conditions, branches and loops

Follow the loop



The final failed condition check matters when counting execution.

- Built-in `&&` and `||` short-circuit. A skipped operand has no side effects.
- `3&&4` is true; `3&4` is 0; `3|4` is 7. Truth tests and bit operations are different.
- A switch chooses an entry label, then statements can fall through. Later cases do not re-test the selector. Case values must be distinct constant integral values after conversion.
- `for`: initialize once -> condition -> body -> update -> condition.
- `while` tests before entry; `do/while` tests after the body.
- `break` exits the nearest loop/switch. `continue` skips the rest of the iteration; a for loop still reaches its update.

03 Arrays, C strings and records

See the storage



The boundary is not another element. A string terminator is stored, even when invisible.

- A four-element array has readable indices 0 through 3. One-past is a boundary, not an extra element.
- `int a[4]={7};` gives 7,0,0,0. An ordinary uninitialized local `int a[4];` has no such guarantee.
- A 2D array is an array of row arrays. `grid[row][col]` selects within a row.
- `char name[]="Ada";` needs four chars, including the null terminator. Its text length is three.
- A struct groups named fields; padding can make its size larger than the sum of field sizes. C++ structs can have methods and inherit.
- `p->field` means `(*p).field` when p is a valid pointer to the object.

Locate names. Build the program.

04 Functions, scope and storage

Follow three calls

NAME VISIBILITY ≠ OBJECT DURATION

ordinary local	call 1: 4	call 2: 4	call 3: 4
local static	call 1: 4	call 2: 5	call 3: 6

Both names stay local. Only the static object retains its changed value.

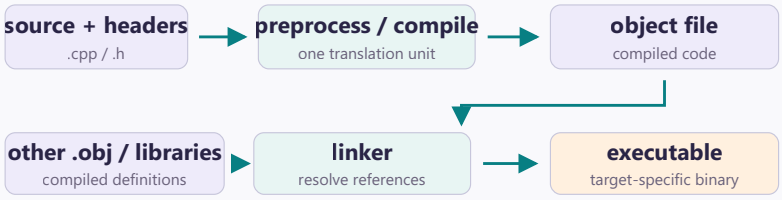
Visibility and duration are different dimensions of the same program.

- Declaration: what a function is. Definition: its body. A suitable declaration must be visible before a call.
- Scope controls where a name is usable. Storage duration controls how long its object exists.
- An ordinary local starts again on each call. A local static initializes once and retains its state across calls in the same run.
- A local pointer can have automatic storage while its target has dynamic storage. Not every variable written without new is a stack local.

05 Declarations, files and the build pipeline

Follow the artifacts

SOURCE TEXT → COMPILED PIECES → RUNNABLE PROGRAM



A header declares the interface; compiled definitions supply the linked behavior.

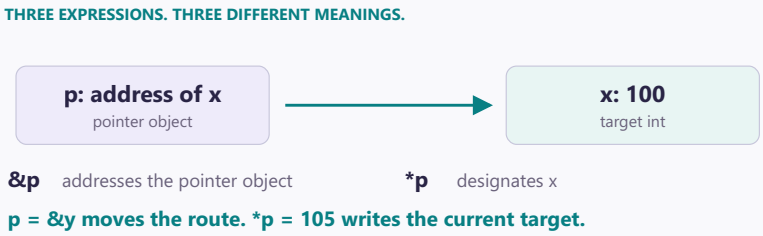
Header declarations -> preprocess/compile -> object files -> link -> executable.

- A prototype allows checked calls; it does not supply the required implementation.
- No visible declaration: compilation problem. Required definition missing from linked code: commonly a linker problem.
- `void Weapon::attack(){...}` defines a member. A free `void attack(){...}` is a different function.
- Include guards prevent repeated inclusion from redefining the class in one translation unit.
- An object file is compiled code, not an instance of a class.

Follow routes and aliases.

06 Pointers and array boundaries

Name the route and target

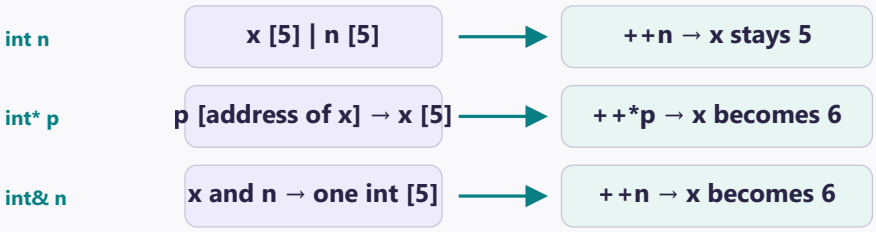


Addresses are symbolic routes here, not measured machine addresses.

- `int* q=p;` copies an address; both pointers can reach the same target. `q=&y;` changes q alone; `*q=9;` changes its current target.
- `p+1` advances one element, not one byte. `p-q` counts elements within the same array or its one-past positions.
- One-past may be formed but not dereferenced. Null, uninitialized and dangling pointers also cannot be safely dereferenced.
- `*p++` means `*(p++)`; `(*p)++` changes the pointed-to value.
- `sizeof(a)` on an actual array measures the complete array. In a parameter declared `int a[]`, a is adjusted to a pointer parameter.

07 References and parameter passing

Compare the three calls



A copied pointer is separate; its target can still be the caller's original object.

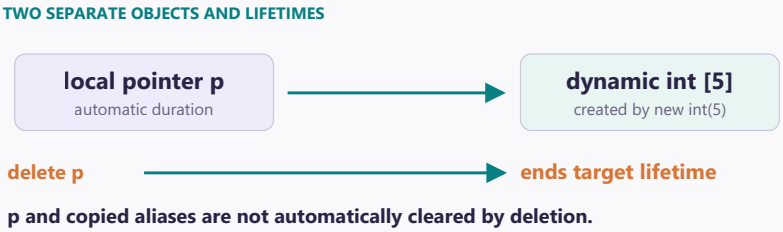
Parameter	What the parameter represents	Effect of shown mutation
`int n`	separate int value	`++n` changes the copy
`int* p`	separate pointer, copied address	`++*p` changes caller's target
`int& n`	alias for caller's object	`++n` changes caller's object

A reference must bind to a suitable live object. It cannot reseat: after `int& r=x;`, `r=y;` copies y's value into x. A reference can still dangle if the referent dies. Passing a pointer by value does not prevent pointee mutation.

Own the lifetime. Unwind the calls.

08 Callbacks and dynamic allocation

Separate lifetimes

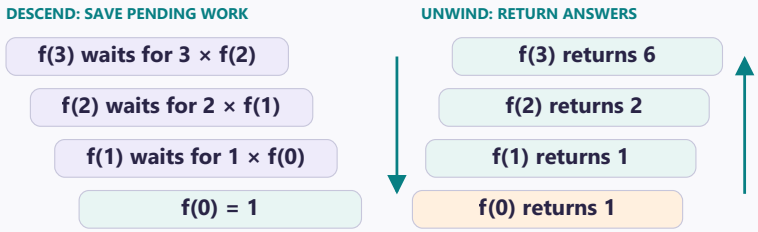


Destroying a local pointer does not automatically destroy a dynamic target.

- `int (*f)(int,int)=add;` is a function pointer. `int* f(int,int);` declares a function returning an int pointer.
- `f(2,3)` invokes the current compatible target. A function-pointer array stores selectable behaviors.
- `new int(5)` initializes to 5; `new int{}` initializes to zero; plain `new int` does not zero-initialize that scalar.
- Pair `new` / `delete` and `new[]` / `delete[]`.
- Deletion ends the target lifetime; pointer aliases are not automatically cleared. Losing the owning address can leak storage.

09 Recursion and the call stack

Down the stack and back



Read down for calls and up for returned answers. Four factorial calls are active at the deepest point.

Specify the input domain, base case and progress toward the base. The simple factorial/power examples require nonnegative inputs and representable results.

There are four factorial calls here, excluding main. Each call preserves its own n and pending work. A logically terminating recursion can still overflow its result type. Work after a recursive call means the shown form is not tail recursion.

Expose behavior. Construct state.

10 Objects, encapsulation and abstraction

State behind an interface

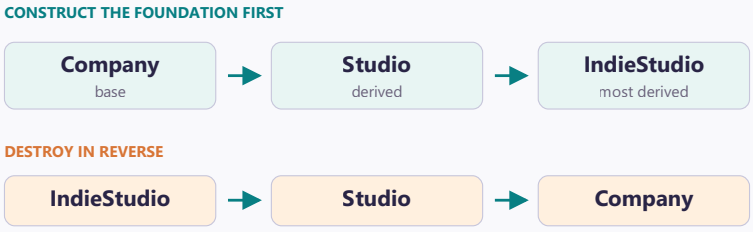


Expose an operation that preserves valid state rather than unrestricted field changes.

- Class = type; object = instance. Non-static state belongs to each object.
- Encapsulation controls state changes through permitted operations. A validating setter can preserve an invariant; a getter need not imply a setter.
- Abstraction exposes the useful interface while hiding irrelevant implementation detail.
- `class` defaults to private member/base access; `struct` defaults to public. Both support C++ class features.
- Private is class/friend access; protected also supports qualifying derived contexts; public is generally available through an appropriate object/interface.

11 Constructors, destructors and lifetime

Build, then dismantle



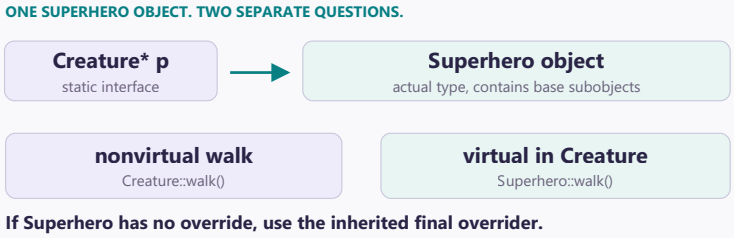
This is a lifetime timeline, not a promise about exact byte layout.

- Constructor: class name, no return type. Destructor: `~ClassName()`, no return type or ordinary parameters.
- Declaring an ordinary constructor can suppress the implicit default constructor. A zero-argument construction must have a viable constructor.
- Base initializes first; members initialize in declaration order; constructor body runs afterward.
- Required base arguments belong in the initializer list: `Studio():Company(5){}`.
- Normal destruction reverses construction. Deleting a derived object through an owning base pointer requires suitable virtual destruction in the ordinary course examples.

Choose behavior. Copy resources.

12 Inheritance and virtual dispatch

Choose the implementation



Static type controls the visible interface. A virtual call uses the actual object's final overrider.

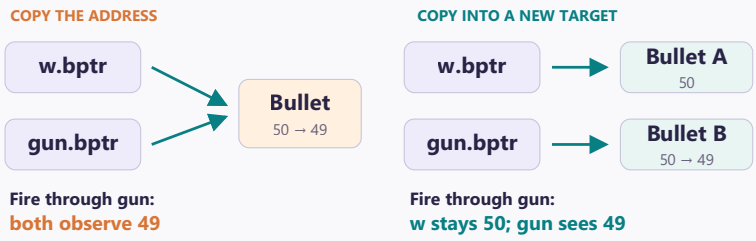
The static pointer/reference type controls the visible interface. A matching virtual call uses the actual object's final overrider. A base view does not transform or slice the original object; a separate base-value copy can slice.

Virtual walk starts at	Creature* to Superhero	Human* to Superhero
Nowhere	Creature::walk	Human::walk
Creature	Superhero::walk	Superhero::walk
Human only	Creature::walk	Superhero::walk

If Superhero has no matching override, the inherited final overrider can be Human. `override` checks the relationship. A class with a remaining pure virtual operation is abstract: its type and pointers are usable, but it cannot be instantiated.

13 Copying and ownership

Sharing versus independence



Equal values can live in different targets. Shared addresses reach the same target.

Code or action	Meaning
`C b(a);` or `C b=a;` at declaration	construct a new b
`b=a;` after b already exists	copy assignment
copy a raw pointer member	copy address; share target
allocate `new T(*other.ptr)`	independent target copy

Implicit copying is memberwise. A class member follows its own copy semantics. Owning raw pointers need consistent copy construction, assignment and destruction; one custom copy constructor is not enough. Return-by-value does not guarantee a fixed copy count because elision/moves matter.

Define operations and relationships.

14 Overloading, overriding and Complex arithmetic

Keep the components aligned

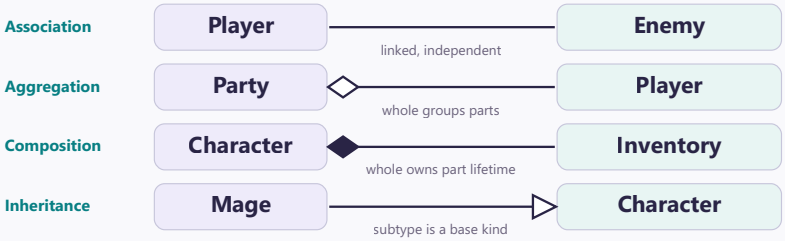


Each lane has a defined meaning; C++ does not guess which fields to add.

- Overloading chooses a signature; overriding supplies an implementation of an inherited virtual operation.
- Parameter names and return type alone do not distinguish ordinary overloads.
- Complex addition combines corresponding real/imaginary components into a result. A non-mutating + leaves inputs unchanged.
- Prefix: `operator++()`. Postfix: `operator++(int)`, normally saving and returning the old value.
- A class defines its own increment meaning. Operator overloading cannot change precedence or invent a new operator symbol.

15 UML and object relationships

Read the relationship symbols



The diamond sits at the whole; the triangle points at the general base.

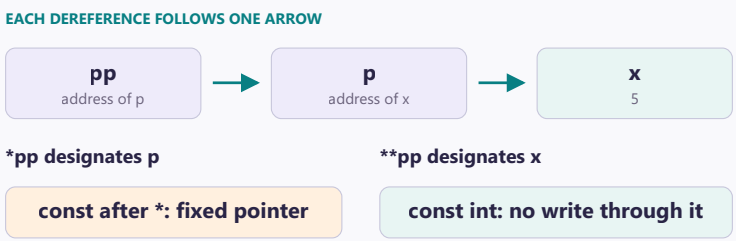
Relationship	Recognition rule	Example policy
Association	solid link	Player targets an independent Enemy
Shared aggregation	hollow diamond at whole	Party groups independently living Players
Composition	filled diamond at whole	Character owns its Inventory lifetime
Inheritance	hollow triangle points to base	Mage is a Character

Visibility: + public, - private, # protected. Multiplicity: 0..1 optional one; * zero or more. Choose ownership from the stated model, not from nouns alone. Nonvirtual diamond inheritance can introduce two copies of a common base; virtual inheritance is different from virtual function dispatch.

Follow types. Recognize shared state.

16 pointer chains, const and void*

Follow one arrow at a time

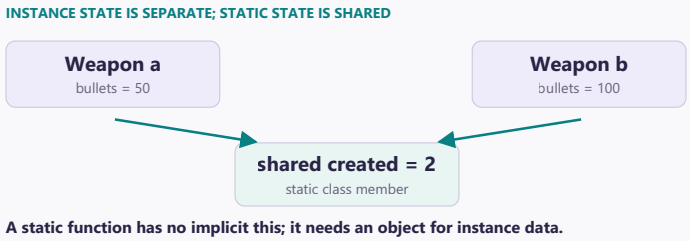


Follow the declared types and live objects at each step; a cast does not create a new reality.

- `int** pp=&p;` : *pp designates p; **pp designates p's int target.
- `int* const p` cannot reseat; `const int* p` cannot write through p; `const int* const p` restricts both.
- A pointer-to-const does not freeze a mutable object through every other access path.
- void* needs a suitable actual typed view before dereference. A cast does not transform the stored object.
- Pointer + pointer, pointer multiplication/division, and `p-=q` for two pointers are ill-formed.

17 instance and static members

Per object versus shared



This counter records the shown construction path; it is not automatically a live-object count.

Non-static fields are per object. A static data member is class-shared state. A static member function has no implicit this, so it needs an explicit instance to reach non-static state. A constructor counter counts only the construction paths implemented; it does not automatically count all copies or live objects.

Final pass

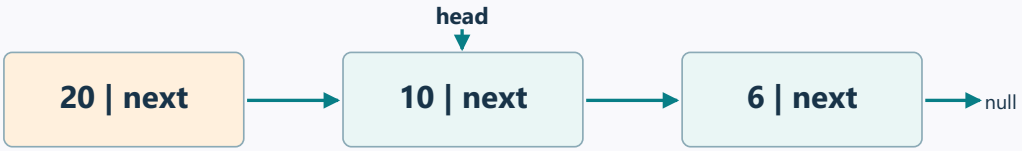
Draw addresses separately from values. Separate initialization from assignment. Write the static type and actual type before a virtual call. Check bounds and lifetime before calculating a number.

Follow links. Preserve ownership.

Linked lists: follow the links

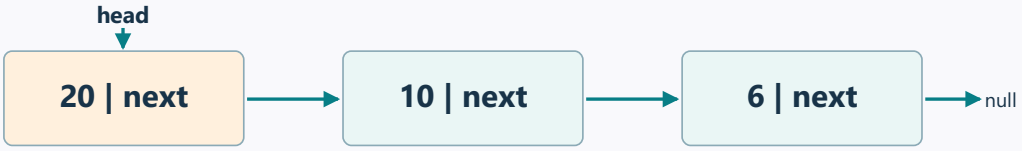
Read the pointer writes

1. Connect new.next to the old head



new node 20 retains the route to 10

2. Move head to the new node



Reachable order: 20 → 10 → 6 → null

A logical chain shows reachability. Connect next before replacing head; physical storage positions are irrelevant here.

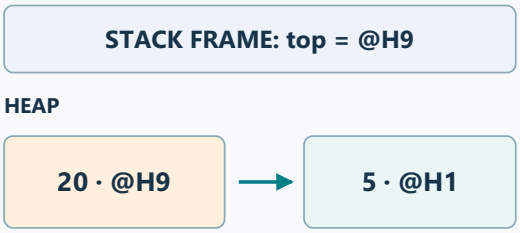
Head is the entry point; a cursor is temporary. Prepend: new.next=old head, then head=new. Append with head only stops at the tail after handling empty. Search tests current, not current->next, so it visits the tail. Null means no node. Doubly linked neighbors must agree in both directions. Save next before delete. Head-only append/search O(n), prepend O(1); tail makes append O(1). Nodes occupy separate allocations and have link overhead.

Stacks and undo / redo

Latest arrival first

SAME ARRIVALS: 5 THEN 20

STACK · LIFO

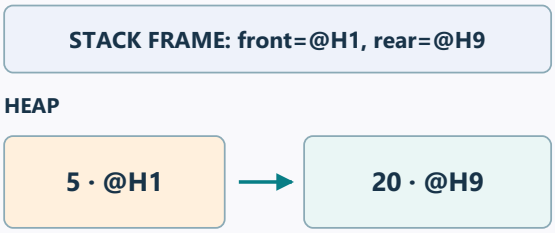


pop removes 20

top moves to @H1

ADT stack nodes are not runtime call frames.

QUEUE · FIFO



dequeue removes 5

front moves to @H9

Final dequeue clears both endpoints.

A user-defined stack can own heap nodes while its top pointer is local. The runtime stack serves active calls.

LIFO/FILO: push and pop at top. Push 5,20 then pop removes 20. Initialize top=nullptr. The custom Node* pop detaches and transfers ownership; caller deletes it. std::stack::pop is void; top first. Undo reverses the latest action and transfers it to redo. A new branch clears redo in our policy. Heap-linked stack nodes are different from runtime call frames.

Order arrivals. Grow sorted regions.

Queues and circular buffers

Oldest arrival first

SAME ARRIVALS: 5 THEN 20

STACK · LIFO

STACK FRAME: top = @H9

HEAP

20 · @H9 → 5 · @H1

pop removes 20

top moves to @H1

ADT stack nodes are not runtime call frames.

QUEUE · FIFO

STACK FRAME: front=@H1, rear=@H9

HEAP

5 · @H1 → 20 · @H9

dequeue removes 5

front moves to @H9

Final dequeue clears both endpoints.

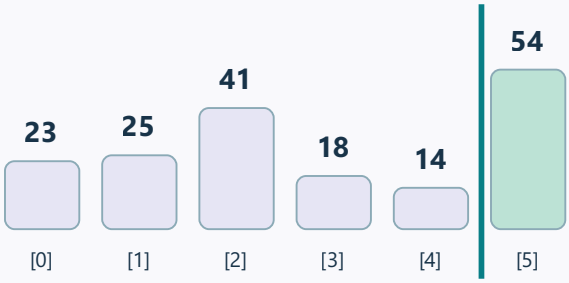
FIFO identifies order of service; a linked queue keeps front and rear. Ring storage instead reuses array slots.

FIFO: enqueue at rear, dequeue at front. Both endpoints start null. Removing the final node clears rear too. Both-endpoint linked operations are O(1). Ring example uses read/write indices plus count: empty 0, full capacity. Advance modulo capacity; refuse invalid operations. Equal indices alone do not distinguish full from empty in this policy.

Bubble and insertion sort

Where is the sorted region?

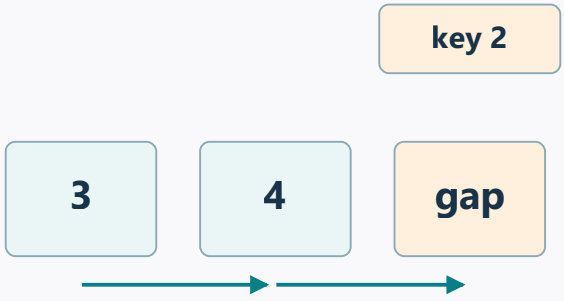
BUBBLE · grow a fixed suffix



Only 54 is guaranteed fixed.

The other bars may still be out of order.

INSERTION · save a key, open a gap



Shift 4, then 3, right into the gap.

Place key 2 → [2, 3, 4]

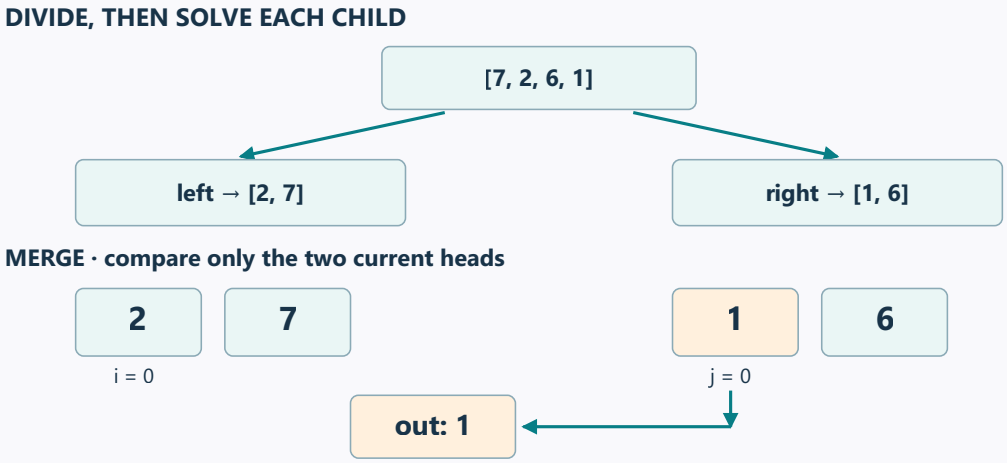
Color marks a guarantee, not decoration: fixed suffix for bubble, sorted prefix for insertion.

Bubble fixes the maximum at the right per pass; neighbor access requires j+1<end. Insertion saves key, shifts larger prefix values, fills the gap; guard j>0 first. Fixed-pass bubble: $\Theta(n^2)$ all orders. Early-exit bubble: $\Theta(n)$ best, $\Theta(n^2)$ worst. Insertion: $\Theta(n)$ best, $\Theta(n^2)$ worst. Both shown iterative forms use O(1) auxiliary storage. Strict > comparisons keep equal elements in order.

Split the work. Merge the results.

Merge sort: split, return, combine

Two sorted lanes, one output



Take 1 from right; advance j only. When a lane ends, append ALL the other tail.

Finish left, then right; combine the two sorted runs. The independent cursors track consumption, not erasure.

Return empty/singleton unchanged. Split n into floor(n/2) and the remainder. Sort left, then right, then merge. Compare only non-exhausted heads; advance only the consumed cursor; append the entire remainder. Left-on-equality preserves stability. Recursion tree contains all work; active stack is one path. Θ(n log n) time, conventional Θ(n) merge workspace, O(log n) call depth. Iterative merge sort is also valid.